Basic (plug search in as a plain tool first) — Hints

Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python, plus the cases a real project has to handle).

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 1 — The idea, and the exact pieces

Basic Version

Doc06 turned plain functions into tools with `@tool`. Do exactly that to Doc08's search: a function that takes a question, calls `retrieve()`, and returns the found text as one string. Then give that one tool to Doc09's Real-world graph (`think` → `act` → `think`) and ask a coffee question.

First, set up `practice/` once for every Doc10 exercise: copy Doc08's Build Task files and `docs/` folder in, and add a tiny `kb.py` that does `collection = build_vector_store("docs")`. Every exercise then does `from kb import collection` and `retrieve(collection, query)` — which returns dicts with `text`, `source_file`, `chunk_index` and `score` (a distance: lower is closer).

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

The exact pieces:

- `@tool` on `def search_docs(query: str) -> str:` — its docstring is what the model reads to decide when to call it, so say *when* to use it.
- Inside: `retrieve(collection, query)`, then collect each `result["text"]` in a list and `"\n\n".join(...)` it.
- `model.bind_tools([search_docs])` in `think`; in `act`, `search_docs.invoke(call["args"])` for every tool call, sent back as a `{"role": "tool", ...}` message (Doc07).

Then the real-project questions: what should the tool return when nothing is found (an explicit sentence, not an empty string)? What if `retrieve()` raises (catch it and return a sentence the model can read)? How many chunks is too many (pass `k=`)? And if a later approval step needs to show *which files* were used, keep the sources separately from the joined text.

Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

Basic Version

```
@tool search_docs(query):
    results = retrieve(collection, query)
    join every result["text"] into one string, return it

graph (Doc09's Real-world graph):
    think: model with bind_tools([search_docs]) -> its reply
    act:   for each tool call -> search_docs.invoke(args) -> tool message
    route: "act" if the reply has tool_calls, else "finish"

ask: "How hot should the water be when brewing?"
```

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

```
search_docs(query: str) -> str:
    try: results = retrieve(collection, query, k=MAX_RESULTS)
    except Exception as exc: return "Search is temporarily unavailable: ..."
    if not results: return "No relevant documents found."
    join the texts

@dataclass SearchResult: text: str, sources: list[str]
search_docs_with_sources(query) -> SearchResult   (same logic, keeps
                                                   each source_file)
```

One piece to get you unstuck:

```
def join_texts(results: list[dict]) -> str:
    pieces: list[str] = []
    for result in results:
        pieces.append(result["text"])
    return "\n\n".join(pieces)
```

Difference between Basic and Intermediate: Basic makes search a working tool. Intermediate makes it safe (a caught failure, a cap, a clear "nothing found") and keeps the sources for a later approval step.

Back to the exercise · Hint 1 · Hint 2 · Solution

Full solution: Show me the solution